

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №9
дисциплины
«Основы нейронных сетей»
Вариант 10

Выполнил:
Рябинин Егор Алексеевич
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии
Воронкин Роман Александрович

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г

Тема: исследование архитектур автокодировщиков для восстановления изображений

Цель: изучить принципы построения и обучения автокодировщиков (Autoencoder) на примере датасета MNIST, реализовать различные архитектуры (сверточные, полносвязные, с латентным пространством малой размерности), освоить методы поиска аномалий, шумоподавления и визуализации многомерных данных.

Порядок выполнения работы:

Ссылка на репозиторий:

https://github.com/bohemiaaaaa/Lab9_Neural-networks

Pytorch:

Задание 1. Добейтесь на автокодировщике с 2-мерным скрытым пространством на 3-х цифрах: 0, 1 и 3 – ошибки $MSE < 0.034$ на скорости обучения 0.001 на 10-й эпохе.

```
import os
import glob
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms

%matplotlib inline
```

Рисунок 1 – Импорт библиотек

```

def ae_on_epoch_end(epoch, loss, encoder, X_train, y_train):
    print('_____')
    print(f'*** ЭПОХА: {epoch+1}, loss: {loss:.6f} ***')
    print('_____')

    plt.figure(dpi=100)

    with torch.no_grad():
        X_tensor = torch.FloatTensor(X_train)
        predict = encoder(X_tensor).numpy()

    scatter = plt.scatter(predict[:, 0], predict[:, 1], c=y_train, alpha=0.6, s=5)
    plt.legend(*scatter.legend_elements(), loc='upper right', title='Классы')

    paths = glob.glob('*.jpg')
    plt.savefig(f'image_{str(len(paths))}.jpg')
    plt.show()

```

Рисунок 2 – Утилиты

```

transform = transforms.Compose([
    transforms.ToTensor(),
])

train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

X_train_full = train_dataset.data.numpy()
y_train_full = train_dataset.targets.numpy()
X_test = test_dataset.data.numpy()
y_test = test_dataset.targets.numpy()

```

Рисунок 3 – Загрузка данных

```

class Autoencoder2D(nn.Module):
    def __init__(self):
        super(Autoencoder2D, self).__init__()

        # Энкодер
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 512),
            nn.ReLU(),
            nn.BatchNorm1d(512),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.BatchNorm1d(256),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 2)
        )

        # Декодер
        self.decoder = nn.Sequential(
            nn.Linear(2, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 28 * 28),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        reconstructed = self.decoder(z)
        return reconstructed, z

    def encode(self, x):
        return self.encoder(x)

model = Autoencoder2D()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Рисунок 4 – Создание модели и обучение

```

Autoencoder2D(
  (encoder): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=512, bias=True)
    (2): ReLU()
    (3): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): Linear(in_features=512, out_features=256, bias=True)
    (5): ReLU()
    (6): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): Linear(in_features=256, out_features=128, bias=True)
    (8): ReLU()
    (9): Linear(in_features=128, out_features=2, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=2, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=512, bias=True)
    (5): ReLU()
    (6): Linear(in_features=512, out_features=784, bias=True)
    (7): Sigmoid()
  )
)

```

Рисунок 5 – Модель

```

# Преобразование в плоские вектора (784)
X_train_flat = X_train.reshape(-1, 784)
X_train_tensor = torch.FloatTensor(X_train_flat)

train_dataset_tensor = TensorDataset(X_train_tensor, X_train_tensor)
train_loader = DataLoader(train_dataset_tensor, batch_size=256, shuffle=True)

print(f"Форма данных: {X_train_tensor.shape}")
print(f"Готово: {len(train_loader)} батчей по 256 изображений")

```

Рисунок 6 – Преобразование в плоские вектора

```

epochs = 10
losses = []

for epoch in range(epochs):
    epoch_loss = 0.0
    model.train()

    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()
        reconstructed, _ = model(batch_x)
        loss = criterion(reconstructed, batch_x)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(train_loader)
    losses.append(avg_loss)

    if (epoch + 1) % 5 == 0:
        print(f'Эпоха {epoch+1}/{epochs}, Loss: {avg_loss:.6f}')
        ae_on_epoch_end(epoch, avg_loss, model.encode, X_train_flat, y_train)

```

Рисунок 7 – Обучение модели

```

model.eval()
with torch.no_grad():
    X_tensor = torch.FloatTensor(X_train_flat)
    reconstructed, _ = model(X_tensor)
    mse = torch.mean((X_tensor - reconstructed) ** 2).item()

    print(f'MSE после {epochs} эпох: {mse:.6f}')

```

MSE после 10 эпох: 0.028146

Рисунок 8 – Вывод результатов

Tensorflow:

Задание 1. Добейтесь на автокодировщике с 2-мерным скрытым пространством на 3-х цифрах: 0, 1 и 3 – ошибки $MSE < 0.034$ на скорости обучения 0.001 на 10-й эпохе.

```
# Работа с операционной системой
import os

# Отрисовка графиков
import matplotlib.pyplot as plt

# Операции с путями
import glob

# Работа с массивами данных
import numpy as np

# Слои
from tensorflow.keras.layers import Dense, Flatten, Reshape

# Модель
from tensorflow.keras import Model

# Загрузка модели
from tensorflow.keras.models import load_model

# Датасет
from tensorflow.keras.datasets import mnist

# Оптимизатор для обучения модели
from tensorflow.keras.optimizers import Adam

# Коллбэки для выдачи информации в процессе обучения
from tensorflow.keras.callbacks import LambdaCallback
```

Рисунок 9 – Импорт библиотек

```

def ae_on_epoch_end(epoch, logs):
    print('_____')
    print(f'*** ЭПОХА: {epoch+1}, loss: {logs["loss"]} ***')
    print('_____')

    # Получение картинки латентного пространства в конце эпохи и запись в файл
    # Задание числа пикселей на дюйм
    plt.figure(dpi=100)

    # Предсказание энкодера на тренировочной выборке
    predict = encoder.predict(X_train)

    # Создание рисунка: множество точек на плоскости 3-х цветов (3-х классов)
    scatter = plt.scatter(predict[:,0,],predict[:,1], c=y_train, alpha=0.6, s=5)

    # Создание легенды
    legend2 = plt.legend(*scatter.legend_elements(), loc='upper right', title='Классы')

    # Сохранение картинки с названием, которого еще нет
    paths = glob.glob('*.jpg')
    plt.savefig(f'image_{str(len(paths))}.jpg')

    # Отображение. Без него рисунок не отрисуется
    plt.show()

ae_callback = LambdaCallback(on_epoch_end=ae_on_epoch_end)

```

Рисунок 10 – Утилиты

```

# Загрузка датасета
(X_train, y_train), (X_test, y_test) = mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/mnist/11490434/11490434 ————— 0s 0us/step

# Нормировка
X_train = X_train.astype('float32')/255.
X_train = X_train.reshape(-1, 28, 28, 1)

```

Рисунок 11 – Загрузка данных и нормировка


```

# Создание модели автокодировщика с 2-мерным скрытым пространством
from tensorflow.keras.layers import Dense, Flatten, Reshape, Input
from tensorflow.keras import Model
from tensorflow.keras.optimizers import Adam

def create_2d_autoencoder():
    img_input = Input(shape=(28, 28, 1))

    # Энкодер
    x = Flatten()(img_input)
    x = Dense(256, activation='relu')(x)
    x = Dense(128, activation='relu')(x)
    latent_space = Dense(2, name='latent_space')(x)

    # Декодер
    x = Dense(128, activation='relu')(latent_space)
    x = Dense(256, activation='relu')(x)
    x = Dense(28 * 28, activation='sigmoid')(x)
    outputs = Reshape((28, 28, 1))(x)

    model = Model(inputs=img_input, outputs=outputs)
    model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
    return model

autoencoder = create_2d_autoencoder()
encoder = Model(inputs=autoencoder.input, outputs=autoencoder.get_layer('latent_space').output)

```

Рисунок 12 – Создание модели и обучение

```

# Обучение модели
history = autoencoder.fit(
    X_train, X_train,
    epochs=10,
    batch_size=256,
    verbose=1,
    callbacks=[ae_callback]
)

```

Рисунок 13 – Обучение модели

```

# Получение предсказаний и расчет MSE
predictions = autoencoder.predict(X_train)
mse = np.mean((X_train - predictions) ** 2)
print(f'MSE на 10-й эпохе: {mse:.6f}')

```

```

588/588 ————— 1s 2ms/step
MSE на 10-й эпохе: 0.033212

```

Рисунок 14 – Получение предсказания и расчет MSE

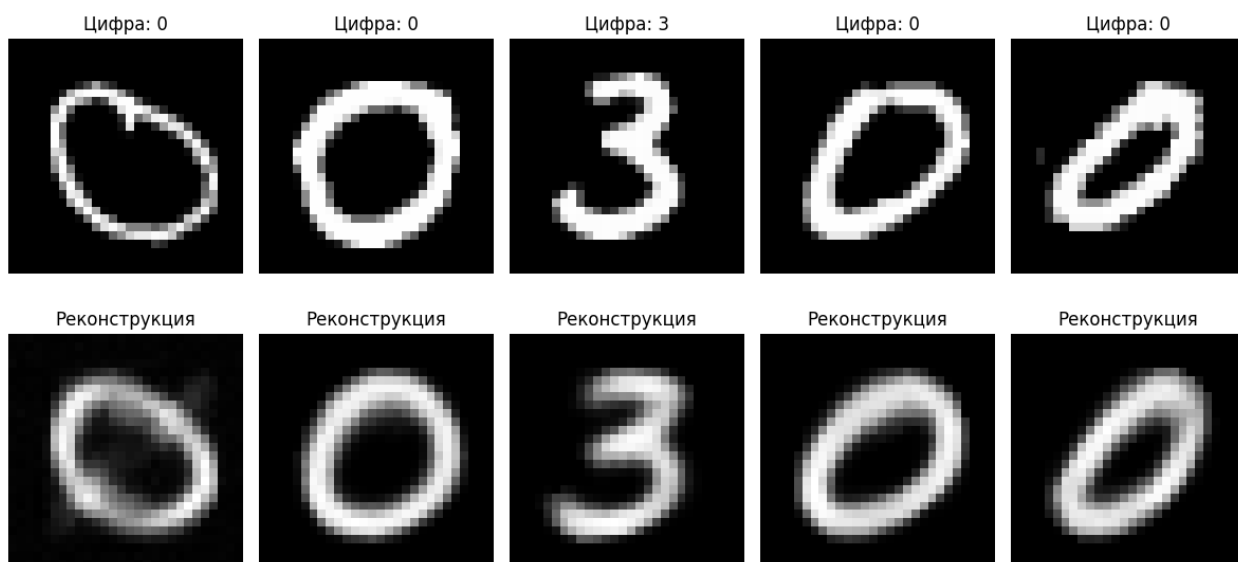


Рисунок 15 – Визуализация оригиналов и реконструкций

Задание 2.

1. Выберите 10 самых красивых по вашему мнению пятерок в тренировочной выборке mnist.
2. Создайте датасет, где объекты – это все пятерки из тренировочной части mnist, а метки – это случайные пятерки из "красивого" набора.
3. Создайте автокодировщик и проверьте, совпадают ли у него размеры выхода и входа.
4. Обучите автокодировщик.
5. Добейтесь ошибки MSE на тренировочной выборке < 0.05 .
6. Посмотрите, как выглядят пятерки из тестовой выборки после обученного автокодировщика.

```

# Для операций с тензорами
import numpy as np

# Для отрисовки
import matplotlib.pyplot as plt

# Для создания модели
from tensorflow.keras.models import Model

# Необходимые слои
from tensorflow.keras.layers import Input, Conv2DTranspose, MaxPooling2D, Conv2D, BatchNormalization

# Слои для латентного пространства модели
from tensorflow.keras.layers import Flatten, Reshape, Dense

# Оптимизатор
from tensorflow.keras.optimizers import Adam

# Для загрузки базы
from tensorflow.keras.datasets import mnist

```

Рисунок 16 – Импорт библиотек

```

# Загрузка датасета
(X_train, y_train), (X_test, y_test) = mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/mnist/11490434/11490434 1s 0us/step

# Нормализация данных
X_train = X_train.astype('float32')/255.
X_test = X_test.astype('float32')/255.

```

Рисунок 17 – Загрузка датасета и нормализация данных

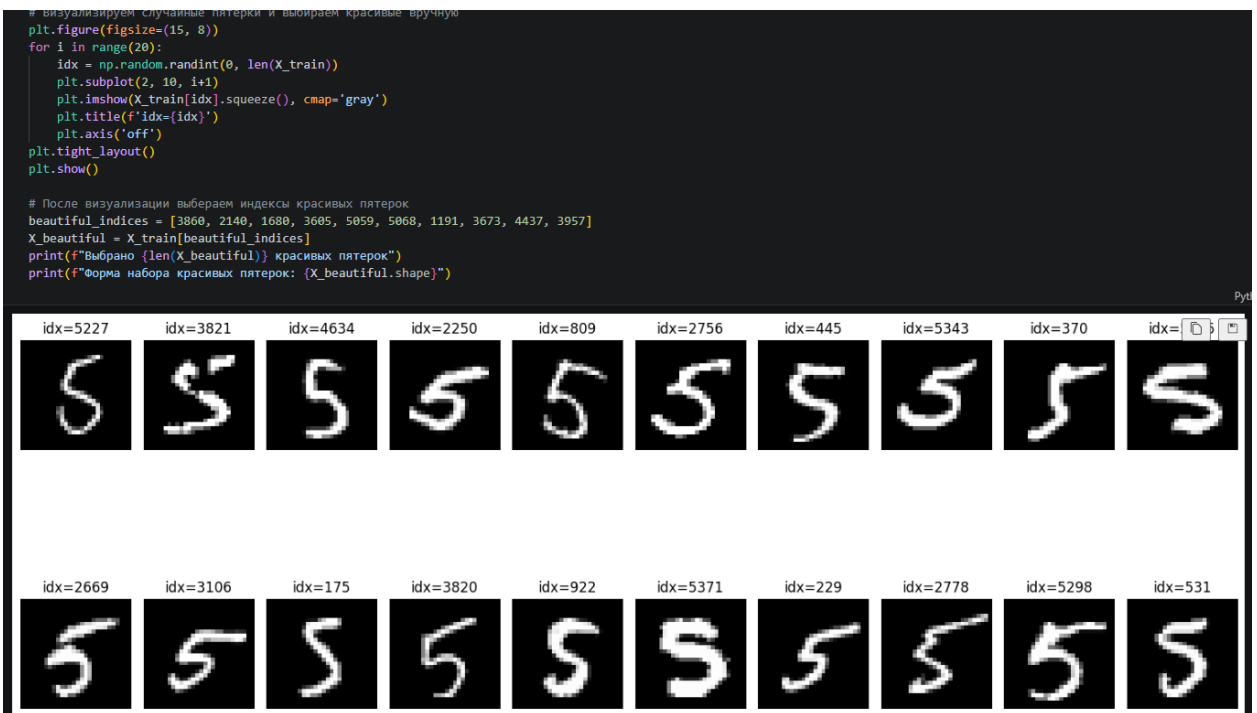


Рисунок 18 – Отбор красивых пятерок

```

def create_autoencoder():
    img_input = Input(shape=(28, 28, 1))

    # Энкодер
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(img_input)
    x = MaxPooling2D((2, 2), padding='same')(x)

    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = MaxPooling2D((2, 2), padding='same')(x)

    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = MaxPooling2D((2, 2), padding='same')(x)

    x = Flatten()(x)
    latent = Dense(256, activation='relu')(x)
    latent = Dense(128, activation='relu')(latent)

    # Декодер
    x = Dense(128, activation='relu')(latent)
    x = Dense(4 * 4 * 128, activation='relu')(x)
    x = Reshape((4, 4, 128))(x)

    x = Conv2DTranspose(64, (3, 3), strides=(2, 2), activation='relu', padding='same')(x)
    x = Conv2DTranspose(32, (3, 3), strides=(2, 2), activation='relu', padding='same')(x)
    x = Conv2DTranspose(1, (3, 3), strides=(2, 2), activation='sigmoid', padding='same')(x)

    from tensorflow.keras.layers import Cropping2D
    x = Cropping2D(cropping=((2, 2), (2, 2)))(x)

    model = Model(inputs=img_input, outputs=x)
    return model

autoencoder = create_autoencoder()
autoencoder.summary()

```

Рисунок 19 – Создание модели

```

autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

history = autoencoder.fit(
    X_train, y_beautiful,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)

```

Рисунок 20 – Обучение модели

```

Epoch 1/20
34/34 ————— 13s 300ms/step - loss: 0.1403 - val_loss: 0.1033
Epoch 2/20
34/34 ————— 9s 254ms/step - loss: 0.0994 - val_loss: 0.0974
Epoch 3/20
34/34 ————— 11s 262ms/step - loss: 0.0759 - val_loss: 0.0625
Epoch 4/20
34/34 ————— 10s 292ms/step - loss: 0.0606 - val_loss: 0.0564
Epoch 5/20
34/34 ————— 10s 295ms/step - loss: 0.0514 - val_loss: 0.0486
Epoch 6/20
34/34 ————— 9s 272ms/step - loss: 0.0454 - val_loss: 0.0442
Epoch 7/20
34/34 ————— 9s 273ms/step - loss: 0.0422 - val_loss: 0.0422
Epoch 8/20
34/34 ————— 11s 305ms/step - loss: 0.0401 - val_loss: 0.0388
Epoch 9/20
34/34 ————— 10s 296ms/step - loss: 0.0351 - val_loss: 0.0337
Epoch 10/20
34/34 ————— 9s 278ms/step - loss: 0.0316 - val_loss: 0.0318
Epoch 11/20
34/34 ————— 9s 264ms/step - loss: 0.0294 - val_loss: 0.0296
Epoch 12/20
34/34 ————— 10s 296ms/step - loss: 0.0269 - val_loss: 0.0262
Epoch 13/20
...
Epoch 19/20
34/34 ————— 9s 251ms/step - loss: 0.0177 - val_loss: 0.0184
Epoch 20/20
34/34 ————— 12s 286ms/step - loss: 0.0164 - val_loss: 0.0170
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Рисунок 21 – Обучение модели

```
train_pred = autoencoder.predict(X_train, verbose=0)
mse_train = np.mean((X_train - train_pred) ** 2)
print(f"MSE: {mse_train:.6f}")
```

MSE: 0.015967

Рисунок 22 – Вывод MSE

Пятерки из тестовой выборки после автокодировщика

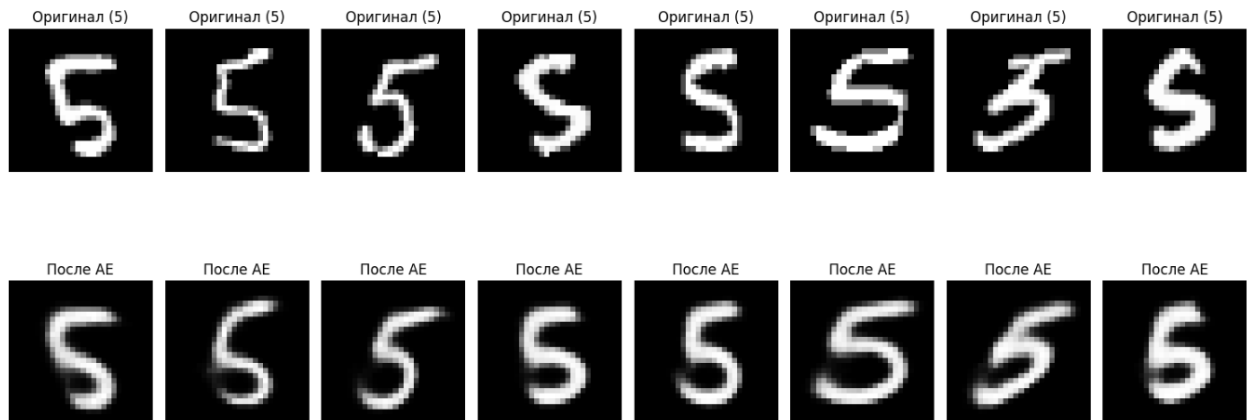


Рисунок 23 – Пятерки из тестовой выборки после автокодировщика

Задание 3. Создайте автокодировщик, удаляющий черные квадраты в случайных областях изображений.

Алгоритм действий:

1. Возьмите базу картинок Mnist.
2. На картинках в случайных местах сделайте чёрные квадраты размера 8 на 8.
3. Создайте и обучите автокодировщик восстанавливать оригинальные изображения из "зашумленных" квадратом изображений.
4. Добейтесь $MSE < 0.0070$ на тестовой выборке

```

# Отображение
import matplotlib.pyplot as plt

# Для работы с тензорами
import numpy as np

# Класс создания модели
from tensorflow.keras.models import Model

# Для загрузки данных
from tensorflow.keras.datasets import mnist

# Необходимые слои
from tensorflow.keras.layers import Input, Conv2DTranspose, MaxPooling2D, Conv2D, BatchNormalization

# Оптимизатор
from tensorflow.keras.optimizers import Adam

```

Рисунок 23 – Импорт библиотек

```

# Загрузка данных
(X_train, y_train), (X_test, y_test) = mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step

```

Рисунок 24 – Загрузка данных

```

# Нормировка данных
X_train = X_train.astype('float32')/255.
X_test = X_test.astype('float32')/255.

# Изменение формы под удобную для Keras
X_train = X_train.reshape((-1, 28, 28, 1))
X_test = X_test.reshape((-1, 28, 28, 1))

```

Рисунок 25 – Нормировка данных

```
def add_black_squares(images, square_size=8):
    images_noisy = images.copy()
    for i in range(len(images)):
        x = np.random.randint(0, 28 - square_size)
        y = np.random.randint(0, 28 - square_size)
        images_noisy[i, x:x+square_size, y:y+square_size, 0] = 0
    return images_noisy

X_train_noisy = add_black_squares(X_train)
X_test_noisy = add_black_squares(X_test)

print(f"X_train_noisy: {X_train_noisy.shape}")
print(f"X_test_noisy: {X_test_noisy.shape}")

# Превращаем в плоские векторы
X_train_flat = X_train.reshape(-1, 784)
X_test_flat = X_test.reshape(-1, 784)
X_train_noisy_flat = X_train_noisy.reshape(-1, 784)
X_test_noisy_flat = X_test_noisy.reshape(-1, 784)

# Визуализация примера
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plt.imshow(X_train_noisy[0].squeeze(), cmap='gray')
plt.title('С черным квадратом')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(X_train[0].squeeze(), cmap='gray')
plt.title('Оригинал')
plt.axis('off')
plt.show()
```

Рисунок 26 – Добавление черных квадратов

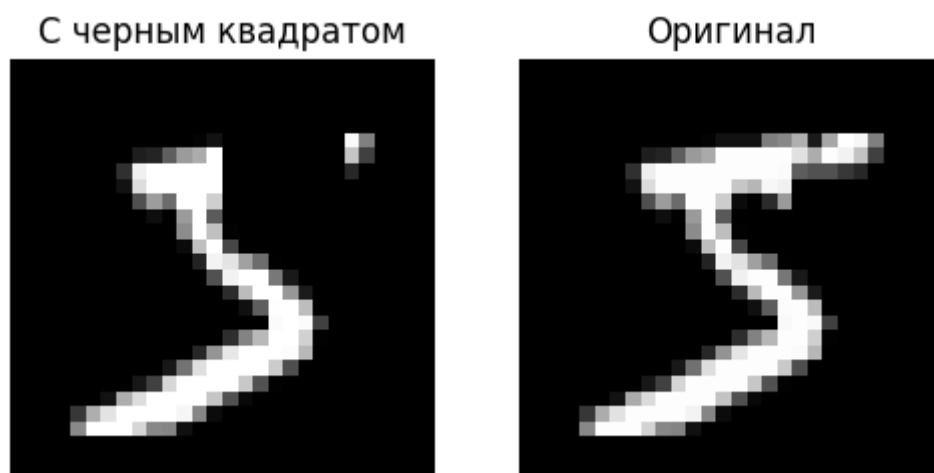


Рисунок 27 – Пример


```

from tensorflow.keras.layers import Dense

inp = Input(shape=(784,))

x = Dense(256, activation='relu')(inp)
x = Dense(128, activation='relu')(x)
x = Dense(64, activation='relu')(x)
x = Dense(128, activation='relu')(x)
x = Dense(256, activation='relu')(x)
out = Dense(784, activation='sigmoid')(x)

autoencoder = Model(inp, out)
autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
autoencoder.summary()

```

Рисунок 28 – Создание модели

```

history = autoencoder.fit(
    X_train_noisy_flat, X_train_flat,
    validation_data=(X_test_noisy_flat, X_test_flat),
    epochs=30,
    batch_size=256,
    verbose=1
)

```

```

Epoch 1/30
235/235 ————— 9s 25ms/step - loss: 0.0590 - val_loss: 0.0353
Epoch 2/30
235/235 ————— 6s 24ms/step - loss: 0.0293 - val_loss: 0.0255
Epoch 3/30
235/235 ————— 6s 27ms/step - loss: 0.0236 - val_loss: 0.0216
Epoch 4/30
235/235 ————— 5s 22ms/step - loss: 0.0207 - val_loss: 0.0196
Epoch 5/30
235/235 ————— 7s 28ms/step - loss: 0.0191 - val_loss: 0.0183
Epoch 6/30
235/235 ————— 10s 27ms/step - loss: 0.0179 - val_loss: 0.0174
Epoch 7/30
235/235 ————— 5s 22ms/step - loss: 0.0170 - val_loss: 0.0168
Epoch 8/30
235/235 ————— 6s 26ms/step - loss: 0.0163 - val_loss: 0.0161
Epoch 9/30
235/235 ————— 6s 25ms/step - loss: 0.0157 - val_loss: 0.0155
Epoch 10/30
235/235 ————— 5s 22ms/step - loss: 0.0152 - val_loss: 0.0153
Epoch 11/30
235/235 ————— 6s 27ms/step - loss: 0.0147 - val_loss: 0.0148
Epoch 12/30
235/235 ————— 5s 22ms/step - loss: 0.0143 - val_loss: 0.0144
Epoch 13/30
...
Epoch 29/30
235/235 ————— 7s 29ms/step - loss: 0.0110 - val_loss: 0.0122
Epoch 30/30
235/235 ————— 6s 24ms/step - loss: 0.0109 - val_loss: 0.0121

```

Рисунок 29 – Обучение модели

```
test_pred_flat = autoencoder.predict(X_test_noisy_flat, verbose=0)
test_pred = test_pred_flat.reshape(-1, 28, 28, 1)
mse = np.mean((X_test - test_pred) ** 2)
print(f"MSE: {mse:.6f}")
```

MSE: 0.012115

Рисунок 30 – Вывод MSE

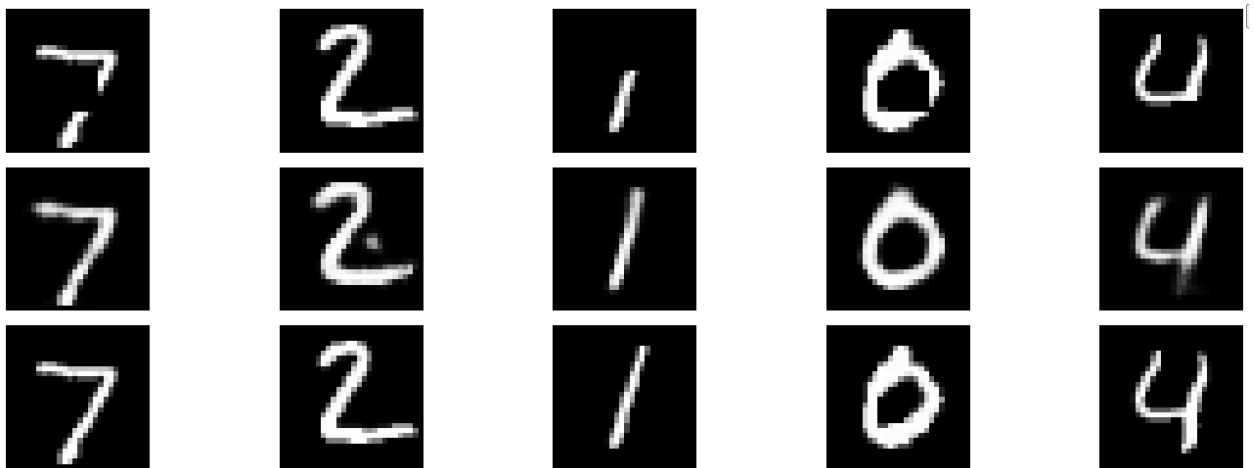


Рисунок 31 – Визуализация работы автокодировщика

Вывод: в ходе лабораторной работы были изучены принципы построения и обучения автокодировщиков на примере датасета MNIST. Реализованы три различные архитектуры: автокодировщик с двумерным латентным пространством для визуализации данных, трансформирующий автокодировщик для преобразования изображений пятерок и шумоподавляющий автокодировщик для удаления черных квадратов.